

Summary and Highlights:

Introduction to DataFrames and Spark SQL

In this lesson, you learned that:

- RDDs are Spark's primary data abstraction partitioned across the cluster's nodes.
- Spark uses directed acyclic graphs (DAGs) to enable fault tolerance. When a node goes down, Spark replicates the DAG and restores the node.
- Transformations undergo lazy evaluation, meaning they are only evaluated when the driver function calls an action.
- A data set is a distributed collection of data that provides the combined benefits of both RDDs and SparkSQL.
- Consisting of strongly typed JVM objects, data sets use DataFrame typesafe capabilities and extend object-oriented API capabilities.
- Data sets work with both Scala and Java APIs. DataFrames are not typesafe. You can use APIs in Java, Scala, and Python. Data sets are Spark's latest data abstraction.
- Spark SQL optimization's primary goal is to improve a SQL query's run-time performance by reducing the query's time and memory consumption, saving organizations time and money.
- Catalyst is the Spark SQL built-in rule-based *query* optimizer. Catalyst performs analysis, logical optimization, physical planning, and code generation.
- Tungsten is the Spark built-in cost-based optimizer for CPU and memory usage that enables cache-friendly computation of algorithms and data structures.
- Basic DataFrame operations are reading, analysis, transformation, loading, and writing.

- You can use a Pandas DataFrame in Python to load a data set and apply the ``printschema``, ``select``, or ``show`` function for data analysis.
- Keep only relevant data for transform tasks and apply functions such as filters, joins, column operations, grouping and aggregations, and other functions.
- Spark SQL consists of Spark modules for structured data processing that can run SQL queries on Spark DataFrames and are usable in Java, Scala, Python, and R.
- Spark SQL supports both temporary views and global temporary views. Use a DataFrame function or an SQL Query and Table view for data aggregation. Spark SQL supports Parquet files, JSON data sets, and Hive tables.

Package/Method	Description	Code Example
<code>appName()</code>	A name for your job to display on the cluster web UI.	<pre> 1. from pyspark.sql import SparkSession 2. spark = SparkSession.builder.appName("MyApp").getOrCreate() </pre>
<code>createDataFrame()</code>	Used to load the data into a Spark DataFrame.	<pre> 1. from pyspark.sql import SparkSession 2. spark = SparkSession.builder.appName("MyApp").getOrCreate() 3. data = [("Jhon", 30), ("Peter", 25), ("Bob", 35)] 4. columns = ["name", "age"] </pre> Creating a DataFrame <pre> 1. df = spark.createDataFrame(data, columns) </pre>
<code>createTempView()</code>	Create a temporary view that can later be	<pre> 1. df.createOrReplaceTempView("cust_tbl") </pre>

Package/Method	Description	Code Example
	used to query the data. The only required parameter is the name of the view.	
fillna()	Used to replace NULL/None values on all or selected multiple DataFrame columns with either zero (0), empty string, space, or any constant literal values.	Replace NULL/None values in a DataFrame <pre>1. filled_df = df.fillna(0)</pre> Replace with zero
filter()	Returns an iterator where the items are filtered through a function to test if the item is accepted or not.	<pre>1. filtered_df = df.filter(df['age'] > 30)</pre>
getOrCreate()	Get or instantiate a SparkContext and register it as a singleton object.	<pre>1. spark = SparkSession.builder.getOrCreate()</pre>
groupby()	Used to collect the identical data into groups on DataFrame and perform count, sum, avg, min, max functions on the grouped data.	Grouping data and performing aggregation <pre>1. grouped_df = df.groupBy("age").agg({"age": "count"})</pre>

Package/Method	Description	Code Example
head()	Returns the first n rows for the object based on position.	Returning the first 5 rows 1. first_5_rows = df.head(5)
import	Used to make code from one module accessible in another. Python imports are crucial for a successful code structure. You may reuse code and keep your projects manageable by using imports effectively, which can increase your productivity.	 1. from pyspark.sql import SparkSession
pd.read_csv()	Required to access data from the CSV file from Pandas that retrieves data in the form of the data frame.	1. import pandas as pd Reading data from a CSV file into a DataFrame 1. df_from_csv = pd.read_csv("data.csv")
pip	To ensure that requests will function, the pip program searches for the	1. pip list

Package/Method	Description	Code Example
	package in the Python Package Index (PyPI), resolves any dependencies, and installs everything in your current Python environment.	
pip install	The pip install <package> command looks for the latest version of the package and installs it.	1. pip install pyspark
printSchema()	Used to print or display the schema of the DataFrame or data set in tree format along with the column name and data type. If you have a DataFrame or data set with a nested structure, it displays the schema in a nested tree format.	1. df.printSchema()
rename()	Used to change the row indexes and the column labels.	1. import pandas as pd

Package/Method	Description	Code Example
		Create a sample DataFrame <pre>1. data = {'A': [1, 2, 3], 'B': [4, 5, 6]} 2. df = pd.DataFrame(data)</pre> Rename columns <pre>1. df = df.rename(columns={'A': 'X', 'B': 'Y'})</pre> The columns 'A' and 'B' are now renamed to 'X' and 'Y' <pre>1. print(df)</pre>
select()	Used to select one or multiple columns, nested columns, column by index, all columns from the list, by regular expression from a DataFrame. select() is a transformation function in Spark	<pre>1. selected_df = df.select('name', 'age')</pre>

Package/Method	Description	Code Example
	and returns a new DataFrame with the selected columns.	
show()	Spark DataFrame show() is used to display the contents of the DataFrame in a table row and column format. By default, it shows only twenty rows, and the column values are truncated at twenty characters.	1. df.show()
sort()	Used to sort DataFrame by ascending or descending order based on single or multiple columns.	Sorting DataFrame by a column in ascending order 1. sorted_df = df.sort("age") Sorting DataFrame by multiple columns in descending order 1. sorted_df_desc = df.sort(["age", "name"], ascending=[False, True])
SparkContext()	It is an entry point to Spark	1. from pyspark import SparkContext

Package/Method	Description	Code Example
	and is defined in org.apache.spark package since version 1.x and used to programmatically create Spark RDD, accumulators, and broadcast variables on the cluster.	Creating a SparkContext <pre>1. sc = SparkContext("local", "MyApp")</pre>
SparkSession	It is an entry point to Spark, and creating a SparkSession instance would be the first statement you would write to the program with RDD, DataFrame, and dataset	Creating a SparkSession <pre>1. from pyspark.sql import SparkSession 1. spark = SparkSession.builder.appName("MyApp").getOrCreate()</pre>
spark.read.json()	Spark SQL can automatically infer the schema of a JSON data set and load it as a DataFrame. The read.json() function loads data from a directory of JSON files where each line of the files is a JSON object. Note that the file offered as	<pre>1. json_df = spark.read.json("customer.json")</pre>

Package/Method	Description	Code Example
	a JSON file is not a typical JSON file.	
spark.sql()	To issue any SQL query, use the sql() method on the SparkSession instance. All spark.sql queries executed in this manner return a DataFrame on which you may perform further Spark operations if required.	<pre> 1. result = spark.sql("SELECT name, age FROM cust_tbl WHERE age > 30") 2. result.show()</pre>
spark.udf.register()	In PySpark DataFrame, it is used to register a user-defined function (UDF) with Spark, making it accessible for use in Spark SQL queries. This allows you to apply custom logic or operations to DataFrame columns using SQL expressions.	Registering a UDF (User-defined Function) <pre> 1. from pyspark.sql.functions import udf 2. from pyspark.sql.types import StringType 3. def my_udf(value): 4. return value.upper() 5. spark.udf.register("my_udf", my_udf, StringType())</pre>
where()	Used to filter the rows from DataFrame based on the given condition. Both filter() and	Filtering rows based on a condition <pre> 1. filtered_df = df.where(df['age'] > 30)</pre>

Package/Method	Description	Code Example
	where() functions are used for the same purpose.	
withColumn()	Transformation function of DataFrame used to change the value, convert the data type of an existing column, create a new column, and many more.	Adding a new column and performing transformations <pre>1. from pyspark.sql.functions import col 2. new_df = df.withColumn("age_squared", col("age") ** 2)</pre>
withColumnRenamed()	Returns a new DataFrame by renaming an existing column.	Renaming an existing column <pre>1. renamed_df = df.withColumnRenamed("age", "years_old")</pre>

Aggregating data	Aggregation is a Spark SQL process frequently used to present aggregated statistics. Commonly used aggregation functions such as count(), avg(), max(), and others are built into Dataframes. Users can also perform aggregation programmatically using SQL queries and table views.
Analyze data using printSchema	In this phase, users examine the schema or the DataFrame column data types using the print schema method. It is imperative to note the data types in each column. Users can apply the select() function to examine data from a specific column in detail.
Catalyst phases	Catalyst analyzes the query, DataFrame, unresolved logical plan, and Catalog to create a logical plan in the Analysis phase. The logical plan evolves into an optimized logical plan in the logical optimization phase. It is the rule-based optimization step of Spark SQL. Rules such as folding, pushdown, and pruning are applicable here. Catalyst generates multiple

	physical plans based on the logical plan in the physical planning phase. A physical plan describes computation on datasets with specific definitions explaining how to conduct the computation. A cost model then selects the physical plan with the least cost. This explains the cost-based optimization step. Code generation is the final phase. In this phase, the Catalyst applies the selected physical plan and generates Java bytecode to run on the nodes.
Catalyst query optimization	Catalyst Optimizer uses a tree data structure and provides the data tree rule sets in the background. Catalyst performs the following four high-level tasks to optimize a query: analysis, logical optimization, physical planning, and code generation.
Catalyst	Within Spark's operational framework, it employs a pair of engines, namely Catalyst and Tungsten, in a sequential manner for query enhancement and execution. Catalyst's primary function involves deriving an optimized physical query plan from the initial logical query plan. This optimization process entails implementing a range of transformations such as predicate pushdown, column pruning, and constant folding onto the logical plan.
Cost-based optimization	Cost is measured and calculated based on the time and memory that a query consumes. Catalyst optimizer selects a query path that results in minimal time and memory consumption. As queries can use multiple paths, these calculations can become quite complex when large datasets are part of the calculation.
Creating a view in Spark SQL	It is the first step in running SQL queries in Spark SQL. It is a temporary table used to run SQL queries. Both temporary and global temporary views are supported by Spark SQL. A temporary view has a local scope. Local scope implies that the view exists within the current Spark session on the current node. A global temporary view exists within the general Spark application. This view is shareable across different Spark sessions.
DAGScheduler	As Spark acts and transforms data in the task execution processes, the DAGScheduler facilitates efficiency by orchestrating the worker nodes across the cluster. This task-tracking makes fault tolerance possible, as it reapplies the recorded operations to the data from a previous state.
DataFrame operations	Refer to a set of actions and transformations that can be applied to a DataFrame, which is a two-dimensional data structure in Spark. Data within a DataFrame is organized in a tabular format with rows and columns, similar to a table in a relational database. These operations encompass a wide range of tasks, including reading data into a

	DataFrame, performing data analysis, executing data transformations (such as filtering, grouping, and aggregating), loading data from external sources, and writing data to various output formats. DataFrame operations are fundamental for working with structured data efficiently in Spark.
DataFrames	Data collection is categorically organized into named columns. DataFrames are conceptually equivalent to a table in a relational database and similar to a data frame in R or Python, but with greater optimizations. They are built on top of the SparkSQL RDD API. They use RDDs to perform relational queries. Also, they are highly scalable and support many data formats and storage systems. They are developer-friendly, offering integration with most big data tooling via Spark and APIs for Python, Java, Scala, and R.
Dataset	The newest Spark data abstraction, like RDDs and DataFrames, provide APIs to access a distributed data collection. They are a collection of strongly typed Java Virtual Machine, or JVM, objects. Strongly typed implies that datasets are typesafe, and the data set's datatype is made explicit during its creation. They offer benefits of both RDDs, such as lambda functions, type-safety, and SQL Optimizations from SparkSQL.
Directed acyclic graph (DAG)	Spark uses a DAG and an associated DAGScheduler to perform RDD operations. It is a graphical structure composed of edges and vertices. Acyclic implies new edges can originate only from an existing vertex. The vertices and edges are sequential. The edges represent transformations or actions. The vertices represent RDDs. The DAGScheduler applies a graphical structure to run tasks using the RDD, performing transformation processes. DAG enables fault tolerance. Spark replicates the DAG and restores the node when a node goes down.
distinct ([numTasks]))	It helps in finding the number of varied elements in a dataset. It returns a new dataset containing distinct elements from the source dataset.
filter (<i>func</i>)	It helps in filtering the elements of a data set basis its function. The filter operation is used to selectively retain elements from a data set or DataFrame based on a provided function (<i>func</i>). It allows you to filter and extract specific elements that meet certain criteria, making it a valuable tool for data transformation and analysis.
flatMap (<i>func</i>)	Similar to map (<i>func</i>) can map each input item to zero or more output items. Its function should return a Seq rather than a single item.
Hive tables	Spark supports reading and writing data stored in Apache Hive.

Java virtual machines (JVMs)	The platform-specific component that runs a Java program. At run time, the VM interprets the Java bytecode compiled by the Java Compiler. The VM is a translator between the language and the underlying operating system and hardware.
JavaScript Object Notation (JSON)	A simplified data-interchange format based on a subset of the JavaScript programming language. IBM® Integration Bus provides support for a JSON domain. The JSON parser and serializer process messages in the JSON domain.
JSON data sets	Spark infers the schema and loads the data set as a DataFrame.
Loading or exporting the data	In the ETL pipeline's last step, data is exported to disk or loaded into another database. Also, users can write the data to the disk as a JSON file or save the data into another database, such as a Postgres (PostgreSQL) database. Users can also use an API to export data to a database, such as a Postgres database.
map (<i>func</i>)	It is an essential operation capable of expressing all transformations needed in data science. It passes each element of the source through a function <i>func</i> , thereby returning a newly formed distributed dataset.
Parquet	Columnar format that is supported by multiple data processing systems. Spark SQL allows reading and writing data from Parquet files, and Spark SQL preserves the data schema.
RDD actions	It is used to evaluate a transformation in Spark. It returns a value to the driver program after running a computation. An example is the reduce action that aggregates the elements of an RDD and returns the result to the driver program.
RDD transformations	It helps in creating a new RDD from an existing RDD. Transformations in Spark are deemed lazy as results are not computed immediately. The results are computed after evaluation by actions. For example, map transformation passes each element of a dataset through a function. This results in a new RDD.
Read the data	When reading the data, users can load data directly into DataFrames or create a new Spark DataFrame from an existing DataFrame.
Resilient Distributed Datasets (RDDs)	A fundamental abstraction in Apache Spark that represents distributed collections of data. RDDs allow you to perform parallel and fault-tolerant data processing across a cluster of computers. RDDs can be created from existing data in storage systems (like HDFS), and they can undergo various transformations and actions to perform operations like filtering, mapping,

	and aggregating. The <i>resilient</i> aspect refers to the Resilient Distributed Datasets (RDDs) ability to recover from node failures, and the <i>distributed</i> aspect highlights their distribution across multiple machines in a cluster, enabling parallel processing.
Schema	It is a collection of named objects. It provides a way to group those objects logically. A schema is also a name qualifier; it provides a way to use the same natural name for several objects and to prevent ambiguous references to those objects.
Spark driver program	A program that functions as software situated on the primary node of a machine. It defines operations on RDDs, specifying transformations and actions. To simplify, the Spark driver initiates a SparkContext linked to a designated Spark Master. Furthermore, it transfers RDD graphs to the Master, the location from which the standalone cluster manager operates.
Spark SQL memory optimization	The primary aim is to improve the run-time performance of a SQL query by minimizing the query time and memory consumption, thereby helping organizations save time and money.
SparkSQL	It is a Spark module that helps in structured data processing. It is used to run SQL queries on Spark DataFrames and has APIs available in Java, Scala, Python, and R.
SQL queries in SparkSQL	Spark SQL allows users to run SQL queries on Spark DataFrames.
String data type	It is the IBM® Informix® ESQL/C data type that holds character data that is null-terminated and does not contain trailing blanks.
Syntax error	If this error is detected while processing a control statement, the remaining statement is skipped and not processed. Any operands in the portion of the statement preceding the error are processed.
toDS() function	Converts data into a typed Dataset for efficient and type-safe operations in PySpark.
Transform the data	In this step of the ETL pipeline, users plan for required dataset transformations, if any. The transformation aims at retaining only the relevant data. Transformation techniques include data filtering, merging with other data sources, or performing columnar operations. Columnar operations include actions such as multiplying each column by a specific number or converting data from one unit to another. Transformation techniques can also be used to group or aggregate

	data. Many transformations are domain-specific data augmentation processes. The effort needed varies with the domain and the data.
Tungsten	Catalyst and Tungsten are integral components of Spark's optimization and execution framework. Tungsten is geared towards enhancing both CPU and memory performance within Spark. Unlike Java, which was initially designed for transactional applications, it seeks to bolster these aspects by employing methods more tailored to data processing within the Java Virtual Machine (JVM). To achieve optimal CPU performance, it also adopts explicit memory management, employs cache-friendly data structures through STRIDE-based memory access, supports on-demand JVM bytecode, minimizes virtual function dispatches, and capitalizes on CPU register placement and loop unrolling.